

M1 : IOT (Internet of Things) SPIFFS & LittleFS

Gilles Menez

Université de Nice – Sophia-Antipolis
Département d'Informatique
email : menez@unice.fr
www : www.i3s.unice.fr/~menez

April 7, 2026: V 1.0

Table of Contents

SPIFFS	4
SPI Flash Filesystem	4
Le partitionnement	5
Pour y mettre ?	6
Un exemple simple de manipulation	7
Uploader en SPIFFS : Plugins IDE Arduino	11
Uploader en SPIFFS : avec VS Code et PlatformIO	13
Un exemple plus complet	14
 LittleFS : Le remplacement	19
Installation de LittleFS	20
Uploader en LittleFS	21
Un exemple en LittleFS	22
TODO	25

SPIFFs

- <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/spiffs.html>
- <https://tttapa.github.io/ESP8266/Chap11%20-%20SPIFFS.html>
- <https://randomnerdtutorials.com/install-esp32-file-system-uploader-arduino-ide/>
- <https://randomnerdtutorials.com/esp32-web-server-spiffs-spi-flash-file-system/>
- <https://tommydesrochers.com/la-meilleure-methode-pour-heberger-une-interface-web-sur-esp32-version-avancee-esp32-ep4/>
- <https://www.onetransistor.eu/2019/12/upload-files-esp8266-esp32-spiffs.html>

SPIFFs : SPI Flash Filesystem I

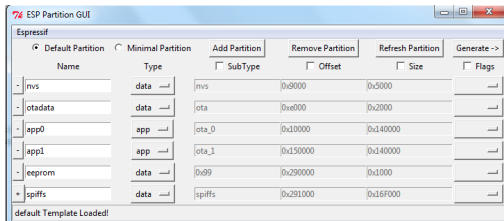
SPIFFS signifie SPI Flash File System (système de fichiers flash SPI).

- ▶ SPI est un protocole souvent utilisé dans le contexte Arduino :
Il signifie Serial Peripheral Interface (SPI)
- ▶ Flash caractérise est la mémoire de votre Contrôleur.
Pour la programmer, il y a une phase dite de "flash" qui correspond à son écriture.
- ▶ Système de fichiers est le terme qui désigne l'organisation de l'espace mémoire qui va autoriser le stockage et la manipulation de "fichiers" dans cette mémoire ... comme sur une carte SD.

Nous pouvons donc stocker des fichiers dans cette mémoire de type SSD sur nos ESP32.

Le partitionnement

La mémoire flash externe peut être "partitionnée" et utilisée pour ... **créer un système de fichiers (i.e Filesystem)** : le **SPIFFS**



- ✓ <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/partition-tables.html>
- ✓ <https://github.com/espressif/arduino-esp32/tree/master/tools/partitions>

Selon la carte de développement que vous choisissez, vous pouvez avoir la possibilité de sélectionner la taille SPIFFS ou aucune option du tout.

- La taille SPIFFS est limitée par la taille globale du flash et la taille du programme.

Pour y mettre ?

L'utilisation d'une partition SPIFFS permet de pouvoir y stocker des fichiers :

- des pages Web,
- des images,
- des fichiers de configuration ou CSS,
- ...

MAIS il y a des limitations car SPIFFS est un système de fichiers "**de base**".

- Contrairement aux systèmes de fichiers traditionnels, le caractère "slash" (/) est autorisé dans les noms de fichiers.
- Il y a une limite de 32 caractères au total pour les noms de fichiers (en fait, seuls 31 sont utilisables).

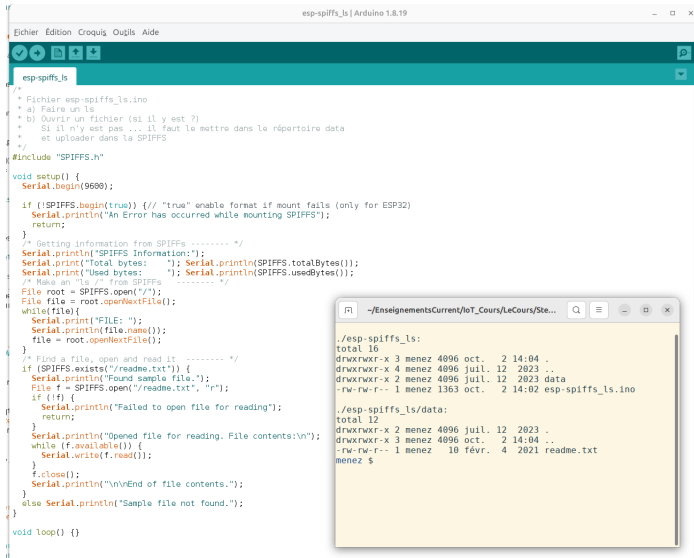
Un exemple simple de manipulation I

```
1  /*
2   * Fichier esp-spiiffs_ls.ino
3   * a) Faire un ls
4   * b) Ouvrir un fichier (si il y est ?)
5   *   Si il n'y est pas ... il faut le mettre dans le répertoire data
6   *   et uploader dans la SPIFFS
7   */
8  #include "SPIFFS.h"
9
10 void setup() {
11   Serial.begin(9600);
12   while (!Serial) {
13     ; // wait for serial port to connect. Needed for native USB
14   }
15
16   if (!SPIFFS.begin(true)) { // "true" enable format if mount fails (only for ESP32)
17     Serial.println("An Error has occurred while mounting SPIFFS");
18     return;
19   }
20
21   /* Getting information from SPIFFS ----- */
22   Serial.println("\nSPIFFS Information:");
23   Serial.print("Total bytes:   "); Serial.println(SPIFFS.totalBytes());
24   Serial.print("Used bytes:      "); Serial.println(SPIFFS.usedBytes());
25   /* Make an "ls /" from SPIFFS ----- */
26   Serial.println("\nMake an \"ls /\n\" =====");
27   File root = SPIFFS.open("/");
28   File file = root.openNextFile();
29   while(file){
30     Serial.print("FILE in directory : ");
31     Serial.println(file.name());
32     file = root.openNextFile();
33   }
34
35 }
```

Un exemple simple de manipulation II

```
36  /* Find a file, open and read it ----- */
37  Serial.println("\nuse/read \"/readme.txt\" file =====");
38  if (SPIFFS.exists("/readme.txt")) {
39      Serial.println("Found sample file.");
40      File f = SPIFFS.open("/readme.txt", "r");
41      if (!f) {
42          Serial.println("Failed to open file for reading");
43          return;
44      }
45      Serial.println("Opened file for reading. File contents :");
46      while (f.available()) {
47          Serial.write(f.read());
48      }
49      f.close();
50      Serial.println("\nEnd of file contents.");
51  }
52  else Serial.println("Sample file not found.");
53  }
54
55  void loop() {}
```


Si on n'oublie d'uploader le répertoire "data" :



```
/*
 * Fichier esp-spiffs_ls.ino
 * a) Faire un ls
 * b) Ouvrir un fichier (si il y est ?)
 * Si il n'y est pas ... il faut le mettre dans le répertoire data
 * et uploader dans la SPIFFS
 */
#include "SPIFFS.h"

void setup() {
  Serial.begin(9600);

  if (!SPIFFS.begin(true)) { // "true" enable format if mount fails (only for ESP32)
    Serial.println("An Error has occurred while mounting SPIFFS");
    return;
  }

  /* Getting information from SPIFFS ----- */
  Serial.println("SPIFFS Information:");
  Serial.print("Total bytes: "); Serial.println(SPIFFS.totalBytes());
  Serial.print("Used bytes: "); Serial.println(SPIFFS.usedBytes());
  /* Make an "ls" from SPIFFS ----- */
  File root = SPIFFS.open("/");
  File file = root.openNextFile();
  while(file){
    Serial.print("FILE: ");
    Serial.println(file.name());
    file = root.openNextFile();
  }

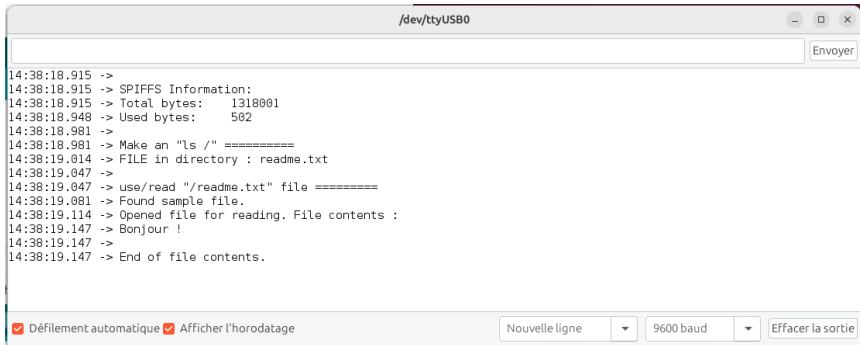
  /* Find a file, open and read it ----- */
  if (SPIFFS.exists("/readme.txt")) {
    Serial.println("Found sample file.");
    File f = SPIFFS.open("/readme.txt", "r");
    if (f) {
      Serial.println("Failed to open file for reading");
      return;
    }
    Serial.println("Opened file for reading. File contents:\n");
    while (f.available()) {
      Serial.write(f.read());
    }
    f.close();
    Serial.println("\n\nEnd of file contents.");
  }
  else Serial.println("Sample file not found.");
}

void loop() {}
```

```
~/EnseignementsCurrent/IoT_Cours/LeCours/Ste...
./esp-spiffs_ls:
total 16
drwxrwxr-x 3 menez 4096 oct.  2 14:04 .
drwxrwxr-x 4 menez 4096 juil. 12 2023 ..
drwxrwxr-x 2 menez 4096 juil. 12 2023 data
-rw-rw-r-- 1 menez 1363 oct.  2 14:02 esp-spiffs_ls.ino

./esp-spiffs_ls/data:
total 12
drwxrwxr-x 2 menez 4096 juil. 12 2023 .
drwxrwxr-x 3 menez 4096 oct.  2 14:04 ..
-rw-rw-r-- 1 menez 10 févr.  4 2021 readme.txt
menez $
```

Cela produit dans la console :



```
14:38:18.915 ->
14:38:18.915 -> SPIFFS Information:
14:38:18.915 -> Total bytes:   1310001
14:38:18.948 -> Used bytes:    502
14:38:18.981 ->
14:38:18.981 -> Make an "ls /" =====
14:38:19.014 -> FILE in directory : readme.txt
14:38:19.047 ->
14:38:19.047 -> use/read "/readme.txt" file =====
14:38:19.081 -> Found sample file.
14:38:19.114 -> Opened file for reading. File contents :
14:38:19.147 -> Bonjour !
14:38:19.147 ->
14:38:19.147 -> End of file contents.
```

☒ Défilement automatique ☒ Afficher l'horodatage

Nouvelle ligne ▼ 9600 baud ▼ Effacer la sortie

Uploader en SPIFFS : Plugins IDE Arduino

L'esptool de Espressif et les autres outils (dont Arduino IDE) basés sur celui-ci ont la **possibilité de télécharger des fichiers** vers SPIFFS.

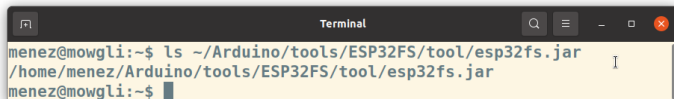
Pour ESP32 dans Arduino IDE, vous utilisez le plugin ESP32FS :

<https://github.com/me-no-dev/arduino-esp32fs-plugin/releases/tag/1.1>

1. Téléchargez l'archive => elle contient un dossier "ESP32FS" avec un jar dedans.
2. Placez ce dossier dans l'emplacement de votre carnet de croquis (sketchbook), sous-dossier "tool" (Créez ce sous-dossier s'il n'existe pas).

Le dossier du carnet de croquis est par défaut dans Documents de l'utilisateur, Arduino pour les utilisateurs Windows.

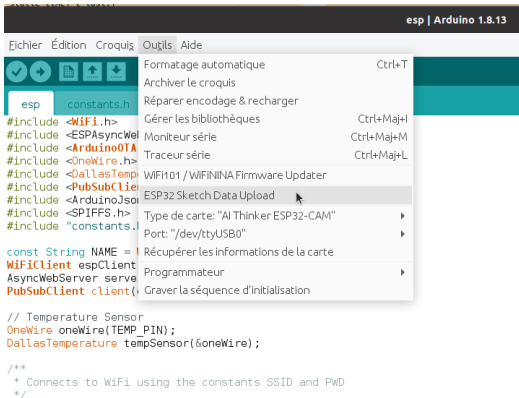
Sous Linux, le jar doit être là !



```
menez@mowgli:~$ ls ~/Arduino/tools/ESP32FS/tool/esp32fs.jar
/home/menez/Arduino/tools/ESP32FS/tool/esp32fs.jar
menez@mowgli:~$
```

Sous Windows ? <https://lededitpro.com/install-esp32-spiiffs-filessystem-uploader-in-arduino-ide/>

Un nouveau champs "Data Upload" apparaît dans le menu "Outils"



Il permet de placer en SPIFFs les fichiers qui se trouve dans le **répertoire data** (et PAS AILLEURS !) qui "accompagne" le sketch :

```
$ cat ../TPs/sketchbook/esp-spiffs_ls/data/readme.txt
Bonjour !
```

Uploader en SPIFFS : avec VS Code et PlatformIO

<https://randomnerdtutorials.com/esp32-vs-code-platformio-spiffs/>

Dans cet [exemple](#), vous trouverez certainement votre bonheur pour manipuler ce SPIFFS !

```
1  /*
2  * Fichier esp-spiffs_test.ino
3  * https://raw.githubusercontent.com/espressif/arduino-esp32/master/libraries/SPIFFS/examples/SPIFFS\_Test/SPIFFS\_Test.ino
4  */
5  #include "FS.h"
6  #include "SPIFFS.h"
7
8  /* You only need to format SPIFFS the first time you run a
9   test or else use the SPIFFS plugin to create a partition
10   https://github.com/me-no-dev/arduino-esp32fs-plugin */
11
12  #define FORMAT_SPIFFS_IF_FAILED true
13
14  void listDir(fs::FS &fs, const char * dirname, uint8_t levels){
15      Serial.printf("Listing directory: %s\r\n", dirname);
16
17      File root = fs.open(dirname);
18      if(!root){
19          Serial.println("- failed to open directory");
20          return;
21      }
22      if(!root.isDirectory()){
23          Serial.println(" - not a directory");
24          return;
25      }
26
27      File file = root.openNextFile();
28      while(file){
29          if(file.isDirectory()){
30              Serial.print(" DIR : ");
31              Serial.println(file.name());
32              if(levels){
33                  listDir(fs, file.name(), levels -1);
34              }
35          }
36          else{
37              Serial.print(" FILE: ");
38              Serial.print(file.name());
39              Serial.print(" SIZE: ");
40              Serial.println(file.size());
41          }
42          file = root.openNextFile();
43      }
44  }
```

```
35     } else {
36         Serial.print(" FILE: ");
37         Serial.print(file.name());
38         Serial.print("\tSIZE: ");
39         Serial.println(file.size());
40     }
41     file = root.openNextFile();
42 }
43 }
44
45 void readFile(fs::FS &fs, const char * path){
46     Serial.printf("Reading file: %s\r\n", path);
47
48     File file = fs.open(path);
49     if(!file || file.isDirectory()){
50         Serial.println("- failed to open file for reading");
51         return;
52     }
53
54     Serial.println("- read from file:");
55     while(file.available()){
56         Serial.write(file.read());
57     }
58     file.close();
59 }
60
61 void writeFile(fs::FS &fs, const char * path, const char * message){
62     Serial.printf("Writing file: %s\r\n", path);
63
64     File file = fs.open(path, FILE_WRITE);
65     if(!file){
66         Serial.println("- failed to open file for writing");
67         return;
68     }
69     if(file.print(message)){
70         Serial.println("- file written");
71     } else {
72         Serial.println("- write failed");
```

```
73     }
74     file.close();
75 }
76
77 void appendFile(fs::FS &fs, const char * path, const char * message){
78     Serial.printf("Appending to file: %s\r\n", path);
79
80     File file = fs.open(path, FILE_APPEND);
81     if(!file){
82         Serial.println("- failed to open file for appending");
83         return;
84     }
85     if(file.print(message)){
86         Serial.println("- message appended");
87     } else {
88         Serial.println("- append failed");
89     }
90     file.close();
91 }
92
93 void renameFile(fs::FS &fs, const char * path1, const char * path2){
94     Serial.printf("Renaming file %s to %s\r\n", path1, path2);
95     if (fs.rename(path1, path2)) {
96         Serial.println("- file renamed");
97     } else {
98         Serial.println("- rename failed");
99     }
100 }
101
102 void deleteFile(fs::FS &fs, const char * path){
103     Serial.printf("Deleting file: %s\r\n", path);
104     if(fs.remove(path)){
105         Serial.println("- file deleted");
106     } else {
107         Serial.println("- delete failed");
108     }
109 }
110
```



```
111 void testFileIO(fs::FS &fs, const char * path){
112     Serial.printf("Testing file I/O with %s\r\n", path);
113
114     static uint8_t buf[512];
115     size_t len = 0;
116     File file = fs.open(path, FILE_WRITE);
117     if(!file){
118         Serial.println("- failed to open file for writing");
119         return;
120     }
121
122     size_t i;
123     Serial.print("- writing" );
124     uint32_t start = millis();
125     for(i=0; i<2048; i++){
126         if ((i & 0x001F) == 0x001F){
127             Serial.print(".");
128         }
129         file.write(buf, 512);
130     }
131     Serial.println("");
132     uint32_t end = millis() - start;
133     Serial.printf(" - %u bytes written in %u ms\r\n", 2048 * 512, end);
134     file.close();
135
136     file = fs.open(path);
137     start = millis();
138     end = start;
139     i = 0;
140     if(file && !file.isDirectory()){
141         len = file.size();
142         size_t flen = len;
143         start = millis();
144         Serial.print("- reading" );
145         while(len){
146             size_t toRead = len;
147             if(toRead > 512){
148                 toRead = 512;
```

```
149         }
150         file.read(buf, toRead);
151         if ((i++ & 0x001F) == 0x001F){
152             Serial.print(".");
153         }
154         len -= toRead;
155     }
156     Serial.println("");
157     end = millis() - start;
158     Serial.printf("- %u bytes read in %u ms\r\n", flen, end);
159     file.close();
160 } else {
161     Serial.println("- failed to open file for reading");
162 }
163 }
164
165 void setup(){
166     Serial.begin(115200);
167     if(!SPIFFS.begin(FORMAT_SPIFFS_IF_FAILED)){
168         Serial.println("SPIFFS Mount Failed");
169         return;
170     }
171     listDir(SPIFFS, "/", 0);
172     writeFile(SPIFFS, "/hello.txt", "Hello ");
173     appendFile(SPIFFS, "/hello.txt", "World!\r\n");
174     readFile(SPIFFS, "/hello.txt");
175     renameFile(SPIFFS, "/hello.txt", "/foo.txt");
176     readFile(SPIFFS, "/foo.txt");
177     deleteFile(SPIFFS, "/foo.txt");
178     testFileIO(SPIFFS, "/test.txt");
179     deleteFile(SPIFFS, "/test.txt");
180     Serial.println("Test complete");
181 }
182
183 void loop(){}

```

LittleFS : Le remplacement ...

SPIFFS a été déprécié par Espressif parce qu'il n'était pas assez résilient et relativement lent.

Selon Espressif,

"LittleFS a été ajouté récemment et se concentre sur des performances plus élevées et la prise en charge des répertoires, mais a un système de fichiers plus élevé et une surcharge par fichier (4K minimum contre 256 octets minimum pour l'unité d'allocation de fichiers de SPIFFS) ".

La structuration LittleFS est donc plus "gourmande" mais elle affiche une meilleure résistance aux coupures de courant.

Ainsi, le système de fichiers ne sera pas corrompu si le courant disparaît pendant une opération d'écriture.

- ▶ Même si un fichier est corrompu, les autres sont en sécurité !

Voir aussi : <https://github.com/RalphBacon/203-SPIFFS-vs-LITTLEFS>

Installation de LittleFS

Vous trouverez la procédure (selon votre OS et votre IDE) dans

- ▶ <https://randomnerdtutorials.com/arduino-ide-2-install-esp8266-littlefs/>
- ▶ <https://randomnerdtutorials.com/esp32-vs-code-platformio-littlefs/>

Pour Linux/Arduino IDE...

- ▶ placer le fichier
<https://github.com/earlephilhower/arduino-littlefs-upload/releases>
- ▶ dans le répertoire "~/arduinoIDE/plugins".

Uploader en LittleFS

1. Sous ArduinoIDE, "Ctrl-Shift-P" et rechercher "Upload Little ..."
2. Attention à ce que le "Serial Monitor" soit fermé ...sinon conflit !
3. Lorsque les data sont uploadées, on peut ensuite uploader le firmware du programme.
 - ▶ Comme pour SPIFFS, ces "data" doivent être dans un répertoire "data".

Attention à ne pas avoir choisi l'option qui écrase la Flash lors du upload ... sinon ce que vous venez de faire va être effacer!

Cet exemple réalise quelques opérations basiques sur un fichier en LittleFS :

► Fichier : esp_littlefs_test.ino

```
1 // Project details: https://RandomNerdTutorials.com/esp32-write-data-littlefs-arduino/
2
3 #include <Arduino.h>
4 #include "FS.h"
5 #include <LittleFS.h>
6
7 /* You only need to format LittleFS the first time you run a
8    test or else use the LITTLEFS plugin to create a partition
9    https://github.com/lorol/arduino-esp32littlefs-plugin
10 */
11 #define FORMAT_LITTLEFS_IF_FAILED true
12
13 int mydata;
14 void writeFile(fs::FS &fs, const char * path, const char * message){
15     /* Write a buffer message if the file path*/
16     Serial.printf("Writing file: %s\r\n", path);
17
18     File file = fs.open(path, FILE_WRITE);
19     if(!file){
20         Serial.println("- failed to open file for writing");
21         return;
22     }
23     if(file.print(message)){
24         Serial.println("- file written");
25     } else {
26         Serial.println("- write failed");
27     }
28     file.close();
29 }
30 void appendFile(fs::FS &fs, const char * path, const char * message){
31     Serial.printf("Appending to file: %s\r\n", path);
32
33     File file = fs.open(path, FILE_APPEND);
```

```
34     if(!file){
35         Serial.println("- failed to open file for appending");
36         return;
37     }
38     if(file.print(message)){
39         Serial.println("- message appended");
40     } else {
41         Serial.println("- append failed");
42     }
43     file.close();
44 }
45 void readFile(fs::FS &fs, const char * path){
46     Serial.printf("Reading file: %s\r\n", path);
47
48     File file = fs.open(path);
49     if(!file || file.isDirectory()){
50         Serial.println("- failed to open file for reading");
51         return;
52     }
53
54     Serial.println("- read from file:");
55     while(file.available()){
56         Serial.write(file.read());
57     }
58     file.close();
59 }
60
61 void setup() {
62     Serial.begin(115200);
63     //while(!Serial);
64     delay(2500); // wait for serial to setup
65
66     // 1) Mount Filesystem (LITTLEFS)
67     if(!LittleFS.begin(FORMAT_LITTLEFS_IF_FAILED)){
68         Serial.println("LittleFS Mount Failed");
69         return;
70     }
71     else{
```

```
72     Serial.println("Little FS Mounted Successfully");
73 }
74
75 // 2) Check if the file already exists to prevent overwriting existing data
76 bool fileexists = LittleFS.exists("/readme.txt");
77 Serial.println(fileexists);
78 if(!fileexists) {
79     Serial.println("File doesn't exist");
80     Serial.println("Creating file...");
81     // Create File and add header
82     writeFile(LittleFS, "/readme.txt", "MY ESP32 DATA \r\n");
83 }
84 else {
85     Serial.println("File already exists");
86     readFile(LittleFS, "/readme.txt"); // Read the contents of the file
87 }
88
89 //Append data to the file
90 mydata = random(0, 1000);
91 appendFile(LittleFS, "/readme.txt", (String(mydata)+ "\r\n").c_str());
92 // Read the contents of the file
93 readFile(LittleFS, "/readme.txt");
94 }
95
96 void loop() {
97     /*
98     mydata = random(0, 1000);
99     appendFile(LittleFS, "/readme.txt", (String(mydata)+ "\r\n").c_str()); //Append data to the file
100     readFile(LittleFS, "/readme.txt"); // Read the contents of the file
101     delay(10000);
102     */
103 }
```


TODO

1. Sur votre PC, vous créez un répertoire data contenant un fichier index.html (le minimum, juste un "hello world" !),
2. Vous l'uploaderez sur l'ESP32,
3. Vous créez un firmware qui affiche dans la console Arduino IDE, les fichiers présents dans LITTLEFS,

Regardez la fonction "listDir" dans :

https://github.com/espressif/arduino-esp32/blob/master/libraries/LittleFS/examples/LITTLEFS_test/LITTLEFS_test.ino